

Nombre: *Jair Patiño*



Programa 12. Regresión OLS

In [ ]:
```
import numpy as np
from math import sqrt
from pprint import pprint
from sklearn import datasets, linear_model, metrics
from sklearn.model_selection import cross_validate, KFold, train_test_split
from sklearn import preprocessing
from sklearn.metrics import make_scorer
import matplotlib.pyplot as plt
```

In [ ]:
```
# Carga de datos.
# load_boston fue eliminado de scikit-learn (>=1.2); se usa California Housing.
datos = datasets.fetch_california_housing()
X = datos.data
y = datos.target
print('Dimensiones de X: ', np.shape(X))
```

In [ ]:
```
# Métricas de evaluación.
metricas = {
    'MAE': 'neg_mean_absolute_error',
    'RMSE': make_scorer(lambda y, y_pred: sqrt(metrics.mean_squared_error(y, y_pred
                        greater_is_better=False),
    'MAPE': make_scorer(lambda y, y_pred: np.mean(np.abs((y - y_pred) / y)) * 100,
                        greater_is_better=False),
    'R2': 'r2',
}
```

In [ ]:
```
# 1) Partición de datos externa
X_training, X_testing, y_training, y_testing = train_test_split(
    X, y, test_size=0.2, random_state=42)
print(np.shape(X_training))
```

In [ ]:
```
# 2) Extracción de características
# 3) Selección de atributos
# 4) Estandarización de los datos de entrenamiento
standardizer = preprocessing.StandardScaler()
stdr_trained = standardizer.fit(X_training)
X_stdr = stdr_trained.transform(X_training)
```

In [ ]:
```
# 5) Construcción del algoritmo de aprendizaje.
reg = linear_model.LinearRegression(fit_intercept=True)
```

In [ ]:
```
# 5.1) Validación cruzada interna
cross_val_results = cross_validate(
    reg, X_stdr, y_training,
    cv=KFold(n_splits=5, shuffle=True, random_state=42),
    scoring=metricas)
pprint(cross_val_results)
```

In [ ]:
```
# 5.2) Entrenamiento con todos los datos de entrenamiento.
model = reg.fit(X_stdr, y_training)
w = model.coef_
print('Model coeficients: \n', w)
w_0 = model.intercept_
print('\nTérmino independiente: ', w_0)
```

---- PREDICCIÓN ----

In [ ]:
```
# 6) Extracción de las características de test
# 7) Selección de los atributos de test
# 8) Estandarización de las características de test
X_test_stdr = stdr_trained.transform(X_testing)
```

In [ ]:
```
# 9) Predicción del conjunto de test
y_pred_test = model.predict(X_test_stdr)
```

In [ ]:
```
# Cálculo de las métricas de evaluación.
MAE = metrics.mean_absolute_error(y_testing, y_pred_test)
MSE = metrics.mean_squared_error(y_testing, y_pred_test)
RMSE = sqrt(MSE)  # squared= fue eliminado en versiones recientes
MAPE = metrics.mean_absolute_percentage_error(y_testing, y_pred_test)
R2 = metrics.r2_score(y_testing, y_pred_test)
print('MAE: %.4f' % MAE)
print('MSE: %.4f' % MSE)
print('RMSE: %.4f' % RMSE)
print('MAPE: %.4f' % MAPE)
print('R2: %.4f' % R2)
```

In [ ]:
```
# Gráfica de realidad vs. predicción.
fig, ax = plt.subplots()
ax.scatter(y_testing, y_pred_test, edgecolors=(0, 0, 0))
ax.plot([y.min(), y.max()], [y.min(), y.max()], 'k--', lw=4)
ax.set_xlabel('Valor real de la clase')
ax.set_ylabel('Predicción')
plt.title("MAE: %.3f MSE: %.3f RMSE: %.3f MAPE: %.3f R2: %.3f" %
          (MAE, MSE, RMSE, MAPE, R2))
plt.grid()
plt.show()
```

GITHUB

- https://github.com/Failedbanjo/machine_learning_2